

Digit Recognizer using GMM and HMM

Artificial Intelligence Assignment 1

Name: _____
Roll Number: _____

April 26, 2026

1 Introduction

The goal of this assignment from my understanding was mainly to understand the concept of GMM and HMM which was taught theoretically in class and how to practically use in real world scenarios. for that first i had to extract required features suitable for such task, then i wrote the implementation code for GMM and HMM from scratch and finally validate and test my models and draw inferences.

2 Feature Extraction

I learnt about the features and feature extraction techniques through LLMs and some youtube videos which explained about mfcc library which uses delta and delta-delta. Audio files were processed with the following pipeline:

- **Pre-emphasis:** $y[n] = x[n] - 0.97x[n-1]$ to boost high frequencies.
- **MFCC:** 20 coefficients, FFT size 512, hop length 160, Hamming window.
- **Deltas:** First and second order derivatives with adaptive window (max 9, min 3 frames).
- **Stacking:** $\text{MFCC} + \Delta + \Delta^2 \rightarrow 60\text{-dim vectors per frame.}$
- **Normalisation:** Z-score standardisation using training set mean and std (plus 10^{-8}).

After extraction, each audio file is represented as a matrix of size $T \times 60$, where T is the number of frames (varies per file) and 60 is the number of features per frame (20 MFCC + 20 delta + 20 delta-delta).

3 My Approaches

3.1 GMM

A separate Gaussian Mixture Model (diagonal covariance, 8 components) is trained per digit using EM from scratch.

Initialization

- Mixing weights: uniform $\pi_k = 1/8$.
- Means: 8 random training frames (without replacement).
- Variances: all ones $+10^{-6}$.

E-Step Compute responsibilities (posterior probabilities):

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \sigma_k)}{\sum_{j=1}^8 \pi_j \mathcal{N}(x_i | \mu_j, \sigma_j)}.$$

We use Log-sum-exp to ensure numerical stability (and the numerical values don't blast off).

M-Step Update parameters using weighted averages:

$$N_k = \sum_i \gamma_{ik}, \quad \pi_k = \frac{N_k}{N}, \quad \mu_k = \frac{\sum_i \gamma_{ik} x_i}{N_k}, \quad \sigma_k = \frac{\sum_i \gamma_{ik} (x_i - \mu_k)^2}{N_k} + 10^{-6}.$$

Training For each digit $d = 0, \dots, 9$, all frames of all training utterances of that digit are stacked (X_d , shape [total frames, 60]). EM runs for at most 100 iterations, stopping when log-likelihood increases by less than 10^{-6} .

Inference For a test utterance (feature matrix $T \times 60$), the total log-likelihood under each digit-specific GMM is computed. The digit with the highest log-likelihood is returned.

Resulting Dimensions Each GMM stores:

- π : (8,)
- μ : (8, 60)
- σ : (8, 60)

3.2 HMM

A separate discrete Hidden Markov Model

A separate discrete Hidden Markov Model (5 states, 64 symbols) is trained per digit using the Baum-Welch algorithm.

Codebook (K-Means) All training feature frames (60-dim) are clustered into 64 code-words using Euclidean K-means. Each audio frame is replaced by its nearest centroid index, converting each of it into an integer sequence.

HMM Initialization

- Initial state: always state 0 ($\pi = [1, 0, 0, 0, 0]$).
- Transition matrix :

$$A_{i,i} = 0.5, A_{i,i+1} = 0.5 \ (i < 4), \quad A_{4,4} = 1.0.$$

- Emission matrix B : random uniform, row-normalised.

Training (Baum-Welch) For each digit, the EM algorithm runs on all the quantized training sequences of that digit:

- **E-step:** Scaled forward-backward computes state occupation probabilities γ and transition counts ξ .
- **M-step:** Update A and B using accumulated statistics across all sequences.

Convergence is checked by total log-likelihood change (tol 10^{-4} , max 100 iterations).

Inference For a test utterance (quantized sequence), the log-likelihood under each digit-specific HMM is computed via the scaled forward algorithm. The digit with the highest log-likelihood is returned.

Resulting Dimensions Each HMM stores:

- A : (5, 5)
- B : (5, 64)
- π : (5,)

4 Experimental Setup

- **Data split:** Speakers 1–5 are used for training (80%) and validation (20%); speaker 6 is held out for testing.
- **Number of training samples:** Training – 2000 utterances; Testing – 500 utterances.

5 Results

5.1 Accuracy Comparison

Comparison of the validation and test accuracies for both GMM and HMM.

Table 1: Overall accuracy comparison.

Model	Validation Accuracy (%)	Test Accuracy (%)
GMM	97.40	91.60
HMM	93.00	74.20

5.2 Confusion Matrices

Figures 1a and 1b show the confusion matrices for the GMM model on the validation and test sets. Figures 2a and 2b show the corresponding matrices for the Discrete HMM.

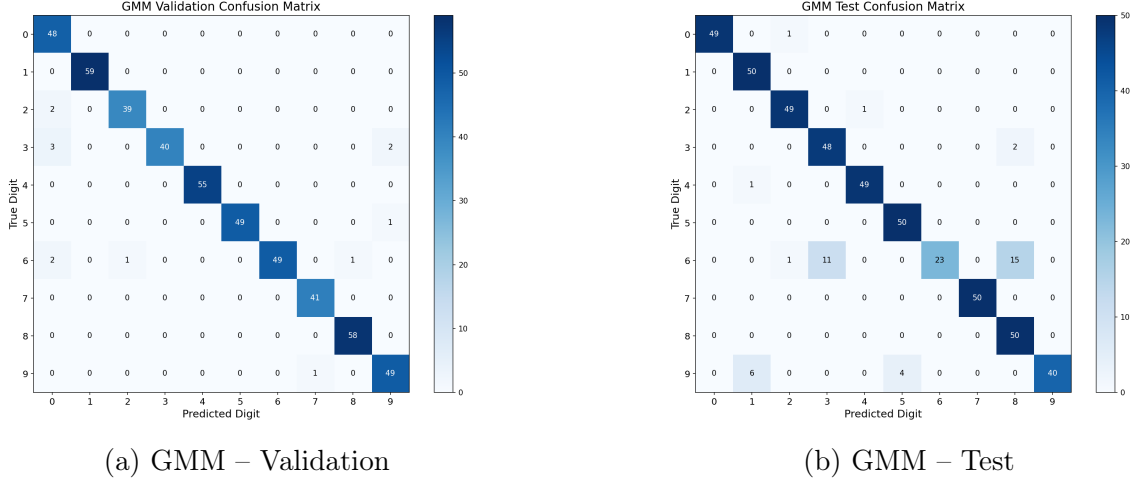


Figure 1: Confusion matrices for GMM.

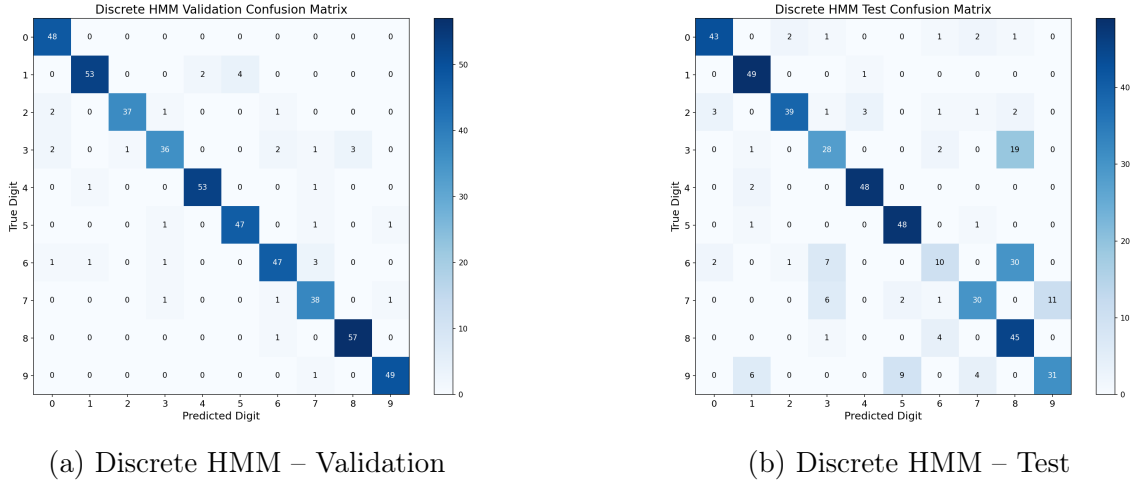


Figure 2: Confusion matrices for Discrete HMM.

5.3 Per-Digit Performance

5.3.1 GMM Per-Digit Accuracy

Table 2: GMM per-digit test accuracy.

Digit	0	1	2	3	4	5	6	7	8	9
Test Acc. (%)	98	100	98	96	98	100	46	100	100	80

5.3.2 Discrete HMM Per-Digit Accuracy

Table 3: Discrete HMM per-digit test accuracy.

Digit	0	1	2	3	4	5	6	7	8	9
Test Acc. (%)	86	98	78	56	96	96	20	60	90	62

6 Discussion

6.1 Comparison of GMM and HMM

The GMM model clearly outperforms the discrete HMM on both validation (97.4% vs 93.0%) and test sets (91.6% vs 74.2%). The large drop in HMM test accuracy indicates overfitting or a mismatch between training and test speakers. The HMM relies on vector quantization, which loses fine discriminative information present in the continuous features, especially for short utterances. GMMs directly model the distribution of feature vectors per digit and therefore generalise better to unseen speakers.

6.2 Insights from Metrics

- **Confusion matrices:** The GMM exhibits few off-diagonal errors; most misclassifications for GMM occur for digits 6 and 9, which are acoustically similar. The HMM shows substantial confusion, particularly for digit 6 (only 20% accuracy) and digit 3 (56% accuracy).
- **Per-digit performance:** GMM maintains high accuracy ($\geq 96\%$) for most digits except digit 6 (46%). The poor performance on digit 6 may be due to its short duration or speaker-dependent articulation. HMM per-digit accuracy is highly variable, with very low scores for digits 3,6,7,9.

6.3 Which Model is More Suited?

For this task, the **GMM** is more suited because:

1. It operates directly on continuous MFCC features without lossy quantization.
2. It captures the overall spectral distribution of each digit without requiring temporal modelling, which is adequate for isolated digit recognition.
3. It generalises better across speakers, as evidenced by the smaller gap between validation and test accuracy.

HMMs would be more beneficial if we had to recognise connected digits or if temporal dynamics were crucial; for isolated digits, the additional complexity of HMMs (and the need for vector quantisation) does not provide an advantage.

7 Conclusion

We implemented and compared GMM and Discrete HMM for isolated digit recognition from speech. The GMM achieved a test accuracy of 91.6%, significantly outperforming the discrete HMM (74.2%), and demonstrated better cross-speaker generalisation. Future work could explore continuous observation HMMs (e.g., using Gaussian mixtures per state) to avoid information loss due to quantisation, and use more sophisticated features (e.g., filter-bank energies) to further improve performance.